use Buildpacks to build containers and run them via Docker Desktop on Apple Silicon computers, but the build process and application startup phase will be slower than usual.

By default, minikube doesn't have access to your local container images, so it will not find the image you have just built for Catalog Service. But don't worry: you can manually import it into your local cluster:

```
$ minikube image load catalog-service --profile polar
```

NOTE YAML is an expressive language, but it can make your coding experience quite bad due to its constraints about spaces or perhaps the lack of support from your editor. When a `kubectl` command involving a YAML file fails, verify that spaces and indentation are used correctly. For Kubernetes, you can install a plugin in your editor to support you while writing YAML manifests, ensuring that you always use the correct syntax, spaces, and indentation. You can find a few plugin options in the README.md file in the repository accompanying this book: https://github.com/ThomasVitale/cloud-native-spring-in-action.

Now that you have a Deployment manifest, let's move on and see how to apply it to your local Kubernetes cluster.

CREATING A DEPLOYMENT OBJECT FROM A MANIFEST

You can apply Kubernetes manifests to a cluster using the kubectl client. Open a Terminal window, navigate to your Catalog Service root folder (catalog-service), and run the following command:

```
$ kubectl apply -f k8s/deployment.yml
```

The command is processed by the Kubernetes Control Plane, which will create and maintain all the related objects in the cluster. You can verify which objects have been created with the following command: